# Memory-Centric Agentic Inference — cycles 13-15

2026-05-11

## Contents



## Memory-Centric Agentic Inference — cycles 13-15

### Abstract

Cycles 13-15 shifted the project from building memory-centric mechanisms to defining the conditions under which those mechanisms should be trusted, selected, or falsified. Cycle 13 completed `M-SEC-1`, a security, provenance, isolation, and retention model for retained agentic state. Cycle 14 completed `M-SYNTH-1`, a cross-milestone architecture synthesis that integrates the previous taxonomy, lifetime, cost, simulation, scheduling, queueing, compression, runtime, calibration, and security artifacts. Cycle 15 began `M-EXP-1`, a measurement-harness milestone that turns the most important deferred constants into production experiment designs.

The main conclusion is conditional rather than universal: future agentic inference infrastructure should expose memory state only at the coarsest boundary that preserves the causal variables needed for reuse value. Conventional request/model/KV serving remains the baseline for controls. A memory-object-aware runtime is justified for RAG-like object reuse when provenance, freshness, isolation, and invalidation gates pass. A trajectory/DAG memory fabric is justified only when branch survival, verifier reuse, tool-output replay, trajectory logs, and durable workspace dependencies retain value after coordination, validation, security, and durable-tail costs.

### Introduction

The project uses three architecture options throughout the package.

Option A is conventional request/model/KV-centric serving. It treats the request, model weights, key-value cache, and short-lived scratch state as the dominant runtime boundary.

Option B is a memory-object-aware runtime. It exposes reusable objects such as retrieved context, prefix cache entries, semantic cache entries, and tool outputs, and it makes placement or retention decisions over those objects.

Option C is a trajectory/DAG-aware memory fabric. It treats long agent runs as directed graphs of state, branches, verifier evidence, replayable tool outputs, trajectory logs, and durable workspace objects.

Cycles 10-12 had already narrowed the technical package: compression/offload no longer claimed unsupported queue-threshold help, the runtime prototype reproduced the A/B/C boundary through trace replay, and the calibration map separated public facts from deferred production constants. Cycles 13-15 used that narrower base to answer a different question: when should retained memory state count as valuable at all?

The answer introduced in cycle 13 is that retained value is conditional on authorized, fresh, isolated, and replayable reuse:

```
AuthorizedReuse(object, actor, source_version, lineage, invalidation_state) = true

SecurityAdjustedValue =
  RetainedValue
  - CoordinationOverhead
  - ValidationOverhead
  - ExpectedSecurityLoss
```

This rule becomes the bridge from the runtime prototype to the final synthesis and then to the measurement agenda.

## Methodology

The reporting sources were the cycle artifacts, the supplied fan-out audit report, the plan of record, the promise ledger, references, generated CSVs, figures, and validation outputs. The direct cycle session IDs supplied for traceability were:

| Cycle | Researcher | Worker | Auditor |
|---|---|---|---|
| 13 | 03c031f2-d730-4a2c-8344-1 | 1 | 1 |
| 14 | f019afaf-7002-4eed-9029-8 | 8 | 1 |
| 15 | 3444472d-a1ac-43ff-8c36-d | d | supplied fan-out audit report |

No callable session-search tool was available in this environment, so the report uses the supplied cycle session identifiers as traceability anchors and relies on the workspace artifacts, ledger records, and audit report for technical content.

The cycle-15 fan-out audit report recorded three branch outcomes:

| Branch | Artifact | Outcome |
|---|---|---|
| Clone 0 | memory-centric-agentic/experiments/tr | d/trajectory_reuse_measurement_plan.md |
| Clone 1 | memory-centric-agentic/experiments/cr | ovenance_overhead_measurement_plan.md |
| Clone 2 | memory-centric-agentic/experiments/ca | d/cache_durable_risk_measurement_plan.md |

## Results

### Cycle 13: Security, Provenance, Isolation, and Retention

Cycle 13 produced `memory-centric-agentic/security_provenance_model.md` and the executable artifact family generated by `scripts/evaluate_security_provenance.py`. The cycle's central decision was to treat security validation as part of architecture selection, not as an external policy layer.

The threat boundary expands across the options. Option A mainly needs cache isolation and cleanup. Option B adds retrieved context, semantic cache entries, and tool outputs, so provenance, source-version freshness, invalidation, tenant/cache-salt partitioning, and cache-poisoning checks become architecture variables. Option C adds branch state, verifier state, trajectory logs, durable workspace objects, and replayable tool outputs, so lineage, replay authorization, verifier evidence integrity, retention policy, and deletion/audit conflicts become part of the runtime boundary.

The generated security model covers all 11 memory-object classes and 10 risk classes. Its key outputs were:

Synthetic security risk score by memory-object class, summed across observed workloads.

Figure 1: Synthetic security risk score by memory-object class, summed across observed workloads.

Mitigation-hook coverage for each security/provenance risk class.

Figure 2: Mitigation-hook coverage for each security/provenance risk class.

| Artifact | Rows | Role |
|---|---|---|
| `data/security_threat_matrix.csv` | 77 | Risk rows by object, workload, option, risk class, required field, failure condition, and mitigation hook. |
| `data/security_risk_scores.csv` | 77 | Synthetic risk, mitigation-overhead, expected-loss, retained-value, and security-adjusted-value rows. |
| `data/security_invalid_trace_fixtures.csv` | 9 | Invalid fixtures that must be rejected or downgraded. |
| `data/security_mitigation_matrix.csv` | 10 | Mitigation hooks covering the risk classes. |
| `data/security_workload_summary.csv` | 6 | Workload-level security-adjusted values and reversal flags. |
| `data/security_special_cases.csv` | 6 | Boundary checks for no sharing, stale reuse, missing provenance, tenant mismatch, verifier tampering, and retention handling. |

The synthetic workload summary showed that security validation can reverse the apparent benefit of richer memory boundaries:

| Workload | Option | Retained value proxy | Validation overhead proxy | Security-adjusted value proxy | Reversal |
|---|---|---|---|---|---|
| RAG | B | 10.8 | 16.527 | -30.631 | true |
| code-agent loop | C | 31.8 | 36.993 | -38.862 | true |
| multi-agent branch/merge | C | 50.0 | 36.993 | -20.662 | true |
| verification-heavy | C | 33.8 | 26.030 | -14.314 | true |

These numbers are not production constants. They are labeled synthetic and are used to show the mechanism: retained memory value can be erased when validation, coordination, and expected security loss are counted.

Cycle 13 also identified trace sufficiency limits. Trace v2 can detect missing provenance, source-version mismatch, invalidation signals, branch/merge lineage errors, verifier identity, and expired durability horizons. It cannot directly encode tenant scope, cache salt, replay actor authorization, verifier evidence hashes, or legal/audit hold state. Those fields became required production extensions for later measurement plans.

Dimensionless retained value, validation overhead, and security-adjusted value by workload/architecture option.

Figure 3: Dimensionless retained value, validation overhead, and security-adjusted value by workload/architecture option.

Final Option A/B/C decision by workload after queueing, compression, calibration, and security gates.

Figure 4: Final Option A/B/C decision by workload after queueing, compression, calibration, and security gates.

**Cycle 14: Final Architecture Synthesis**

Cycle 14 produced `memory-centric-agentic/final_synthesis.md`, `scripts/synthesize_research_agenda.py`, `scripts/plot_synthesis.py`, and four synthesis CSVs. The worker and auditor records state that the synthesis integrated the validated prior milestones into a decision matrix, 12 claims, 12 open risks, and a ranked research agenda. The auditor validated the package after a scoped plot-dispatch patch and confirmed that all three synthesis figures regenerate and are nonblank.

The synthesis decision rule is:

```
ArchitectureChoice(w) =
  argmax over A/B/C of
    visible retained value
  + movement/correctness benefit
  - coordination cost
  - compression/recovery cost
  - validation overhead
  - expected security loss

subject to authorized reuse for every retained object.
```

The final workload decisions were:

| Workload | Final option | Main positive mechanism | Main reversal risk |
|---|---|---|---|
| single-turn chat control | A | No durable, branch, or cross-request retained variables. | None identified; Option A remains dominant. |
| batch/offline control | A | No durable, branch, or cross-request retained variables. | None identified; Option A remains dominant. |
| RAG | B | Retrieved-context and semantic/prefix reuse with provenance and freshness checks. | Security validation overhead, semantic-cache correctness/invalidation cost, and metadata queues. |
| code-agent loop | C | Branch survival, verifier retention, trajectory lineage, and durable workspace reuse. | Replay authorization, verifier integrity, durable latency tails, DAG coordination, and security overhead. |
| verification-heavy | C | Verifier retention, branch state, trajectory lineage, and durable replay. | Verifier evidence validation, replay authorization, DAG overhead, and durable-tail cost. |
| multi-agent branch/merge | C | Branch survival, verifier retention, trajectory lineage, and durable workspace reuse. | Branch replay ambiguity, durable tails, DAG coordination, and security validation overhead. |

The synthesis classified claims by evidence status. The strongest validated claims were that controls collapse to Option

Ranked prototype/research experiments by priority and uncertainty reduction.

Figure 5: Ranked prototype/research experiments by priority and uncertainty reduction.

Robust, sensitive, and speculative claims mapped against falsification difficulty.

Figure 6: Robust, sensitive, and speculative claims mapped against falsification difficulty.

A, unsafe reuse must be rejected before retained value is counted, compression/offload must be representation-safe before byte savings count, and security validation is part of architecture selection. The sensitive claims concerned Option B and Option C because they depend on unmeasured production constants. The speculative claims concerned energy/economics and durable multi-agent reuse because per-tier energy, pricing, production trajectory reuse, and durable latency-tail measurements remain absent.

The highest-ranked experiments were:

1. `EXP-TRAJ-REUSE`: measure production reuse of trajectory, verifier, branch, and durable workspace state.
2. `EXP-PROV-OVERHEAD`: measure provenance, source-version, cache-salt, and lineage validation overhead.
3. `EXP-SEMANTIC-CACHE-CORRECTNESS`: measure stale, poisoned, tenant-invalid, or semantically false-positive cache hits.
4. `EXP-QUEUE-HOTPATH`: measure registry, policy, migration, verifier-sync, and preemption queue saturation.
5. `EXP-DURABLE-STORE-TAILS`: measure durable workspace and object-store replay/checkpoint latency tails.

Cycle 14 therefore closed the first full architecture synthesis but did not close the research program. It converted the earlier models into a falsification agenda.

### Cycle 15: Measurement Harness for Deferred Constants

Cycle 15 added `M-EXP-1` to the plan of record. The new milestone converts deferred constants into measurement designs and parent harness CSVs. The cycle focused on the constants most likely to decide whether Option B and Option C survive outside synthetic traces.

The parent measurement harness now contains:

| Artifact | Rows | Contents |
|---|---|---|
| `data/measurement_experiment_specs.csv` | 18 | Experiment rows for DC-005 trajectory reuse, DC-004 semantic-cache risk, and DC-003 durable replay tails. |
| `data/measurement_required_fields.csv` | 22 | Production trace and measurement fields. |
| `data/measurement_thresholds.csv` | 14 | Collapse, downgrade, and forbid thresholds. |
| `data/measurement_claim_update_matrix.csv` | 22 | Claim-update paths. |
| `data/measurement_synthetic_probe_results.csv` | 12 | Synthetic mechanism-probe rows. |
| `data/dc005_merge_verification_results.csv` | 10 | DC-005 merge-readiness checks, all passing. |

The fan-out branches contributed different parts of the measurement agenda.

**DC-005: Production Trajectory Reuse** Clone 0 produced `trajectory_reuse_measurement_plan.md`. The branch defines how to measure whether Option C has real production value from branch survival, verifier-state reuse, tool-output replay, trajectory-log replay, and durable workspace reuse.

The core inequality is:

```
NetTrajectoryValue =
    V_branch
```

```
  + V_verifier
  + V_tool_replay
  + V_trajectory_log
  + V_durable_workspace
  - Q_dag
  - Q_verifier_sync
  - Q_durable_consistency
  - Q_preemption
  - ValidationOverhead_trajectory
  - ExpectedSecurityLoss_trajectory
```

Option C remains justified only when this value beats the relevant Option B or Option A fallback. The parent harness integrated seven trajectory experiments, `TRJ-001` through `TRJ-007`, and four threshold rows: `C_to_B_trajectory_reuse`, `C_to_A_trajectory_reuse`, `p_survive_min`, and `p_verifier_reuse_min`.

The branch explicitly treats authorization and provenance fields as gates for counting value, not as the measured overhead term. That scope boundary is why the verifier checks that DC-006 provenance-overhead rows do not reuse DC-005 experiment IDs or collapse thresholds.

**DC-006: Provenance-Validation Overhead** Clone 1 exited without a merge report, but `provenance_overhead_measuremen` exists. The artifact designs measurement for the overhead of provenance pointer lookup, source-version freshness, tenant/cache-salt isolation, trajectory lineage validation, replay actor authorization, verifier evidence binding, retention hold-state compliance, and summary-pointer recoverability.

The plan defines validation gates `PV-01` through `PV-08` and experiments such as gate microbenchmarks, end-to-end trace replay validation, tenant/cache-salt reuse A/B tests, lineage and replay authorization tests, verifier evidence hash binding, retention hold validation, and summary-plus-pointer recovery validation.

Because the branch exited without a merge report, DC-006 is a gap in parent integration. The artifact is usable source material, but the current parent measurement CSVs contain no DC-006 rows.

**DC-004 and DC-003: Cache/Durable Risk** Clone 2 produced `cache_durable_risk_measurement_plan.md`. The branch covers two deferred constants:

| Deferred constant | Measurement target | Architecture risk |
|---|---|---|
| DC-004 | Semantic-cache correctness and invalidation cost. | RAG Option B fails if raw hits do not become valid safe hits after provenance, freshness, tenant/cache-salt, poisoning, and recovery checks. |
| DC-003 | Remote durable object-store latency distributions for agent state. | Option C durable replay fails if p95/p99 replay tails and dependency fan-in exceed retained value. |

The semantic-cache retained-value expression is:

```
V_semantic_safe =
  P_hit * P_valid_given_hit * C_recompute_avoided
  - C_lookup
  - C_validation
  - C_invalidation
  - C_recovery
  - E[C_wrong_reuse]
```

The durable replay value expression is:

```
V_durable_replay =
  P_replay * C_recompute_avoided
  - L_store_read(p)
```

```
- L_consistency(p)
- C_pointer_validation
- C_reconstruction
- E[C_replay_failure]
```

The parent harness integrated six semantic-cache experiments, `CDR-SEM-001` through `CDR-SEM-006`, and five durable-state experiments, `CDR-DUR-001` through `CDR-DUR-005`. It also integrated five semantic thresholds, `T-SEM-*`, and five durable thresholds, `T-DUR-*`.

**DC-005 Merge Verification** Cycle 15 also added `memory-centric-agentic/experiments/dc005_merge_verification.md`, `tests/verify_dc005_merge_ready.py`, and `data/dc005_merge_verification_results.csv`. The verifier checks that the shared measurement CSVs are parseable, contain `TRJ-001` through `TRJ-007`, retain the required trajectory thresholds, include required fields such as `trajectory_node_id`, `replay_authorization_scope`, `verifier_evidence_hash`, and `retention_hold_state`, and do not let DC-006 overhead rows reuse DC-005 trajectory IDs.

The verifier passed all 10 checks.

## Discussion

Cycles 13-15 changed the research package in three ways.

First, security became a first-order architecture variable. Before cycle 13, retained object value could be discussed as a performance or capacity question. After cycle 13, retained value is counted only when reuse is authorized, fresh, isolated, and replayable. This matters because semantic caches, prefix caches, trajectory logs, and durable workspace state can all look useful under byte or latency metrics while being invalid under provenance or retention rules.

Second, the architecture proposal became a decision rule instead of a fixed recommendation. The synthesis does not claim that every agentic system needs a trajectory/DAG memory fabric. It says that the right boundary is the coarsest one that preserves causal retained-value variables. Controls stay at Option A. RAG-like systems move to Option B only when object reuse survives correctness and validation gates. Agentic branch/verify/durable workloads move to Option C only when trajectory value survives queueing, replay validation, verifier integrity, retention, and durable-tail costs.

Third, the project now has a measurement agenda rather than only synthetic evidence. The most important unresolved quantities are no longer hidden assumptions. They are named deferred constants with experiment IDs, required fields, thresholds, negative tests, and claim-update paths.

The public references still support only part of this package. Existing systems and sources support conventional serving controls, GPU memory/fabric facts, PCIe/NVMe/CXL capability framing, KV memory management, prefix caching, and semantic-cache mechanisms [1]-[14]. They do not supply production agent trajectory reuse distributions, provenance-validation overheads, durable object-store replay tails for agent state, or semantic-cache safe-hit rates under correctness and isolation constraints. That is why cycle 15 focused on measurement design.

## Conclusions and Recommendations

The cycles 13-15 result is a more defensible and more conditional memory-centric architecture thesis:

```
Memory-centric architecture is strongest when
  retained state value
  > coordination cost
  + validation overhead
  + recovery cost
  + expected security loss
  + durable-tail cost.
```

The package now supports three concrete recommendations.

1. Treat memory reuse as a correctness-gated operation. Runtime value calculations should not count reused retrieved context, semantic cache entries, tool outputs, branch state, verifier state, trajectory logs, or durable workspace objects unless the relevant provenance, freshness, isolation, lineage, authorization, verifier, and retention checks pass.

2. Preserve the A/B/C boundary as a design discipline. Option A is still the right baseline for controls. Option B is the right target for object reuse with valid provenance and invalidation behavior. Option C is the right target only for workloads with measurable branch, verifier, trajectory, and durable replay value.
3. Prioritize measurement before expanding the model. The next engineering work should implement the M-EXP-1 harness for DC-005, DC-006, DC-004, and DC-003, then feed measured distributions back into the synthesis matrix.

The open gap is DC-006 parent integration. The provenance-overhead measurement plan exists, but clone 1 exited without a merge report and the current parent measurement CSVs contain no DC-006 rows. The next cycle should either integrate that artifact into the parent harness or explicitly reopen the branch.

## References


[1] NVIDIA, "NVIDIA H100 Tensor Core GPU," NVIDIA Data Center, accessed 2026-05-11. https://www.nvidia.com/en-us/data-center/h100/

[2] NVIDIA, "NVIDIA H200 Tensor Core GPU," NVIDIA Data Center, accessed 2026-05-11. https://www.nvidia.com/en-us/data-center/h200/

[3] NVIDIA, "NVIDIA DGX B200: The Foundation for Your AI Factory," NVIDIA Data Center, accessed 2026-05-11. https://www.nvidia.com/en-gb/data-center/dgx-b200/

[4] NVIDIA, "Introduction to NVIDIA DGX H100/H200 Systems," NVIDIA DGX H100/H200 User Guide, accessed 2026-05-11. https://docs.nvidia.com/dgx/dgxh100-user-guide/introduction-to-dgxh100.html

[5] PCI-SIG, "PCI Express 6.0 Specification," PCI-SIG, accessed 2026-05-11. https://pcisig.com/pci-express-6.0-specification

[6] NVM Express, "Specifications," NVM Express, accessed 2026-05-11. https://nvmexpress.org/specifications/

[7] Compute Express Link Consortium, "CXL Specification," Compute Express Link, accessed 2026-05-11. https://computeexpresslink.org/cxl-specification/

[8] MLCommons, "MLPerf Inference: Datacenter Benchmark," MLCommons, accessed 2026-05-11. https://mlperf.pw/benchmarks/inference-datacenter/index.html

[9] Woosuk Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention," arXiv, 2023. https://arxiv.org/abs/2309.06180

[10] Intel, "Intel Xeon 6 Processors with MRDIMM — Solution Brief," Intel, accessed 2026-05-11. https://www.intel.com/content/www/us/en/content-details/919018/intel-xeon-6-processors-with-mrdimm-solution-brief.html

[11] AMD, "AMD EPYC 9005 Processor Architecture Overview," AMD, accessed 2026-05-11. https://www.amd.com/content/dam/amd/en/documents/epyc-technical-docs/user-guides/58462_amd-epyc-9005-tg-architecture-overview.pdf

[12] vLLM Project, "Automatic Prefix Caching," vLLM Documentation, accessed 2026-05-11. https://docs.vllm.ai/en/latest/design/prefix_caching/

[13] Sajal Regmi and Chetan Phakami Pun, "GPT Semantic Cache: Reducing LLM Costs and Latency via Semantic Embedding Caching," arXiv, 2024. https://arxiv.org/abs/2411.05276

[14] CXL Consortium, "CXL Consortium Releases Compute Express Link 2.0 Specification," Business Wire, 2020. https://www.businesswire.com/news/home/20201110005037/en/CXL-Consortium-Releases-Compute-Express-Link-2.0-Specification


## Appendix: Implementation Details

### Code Organization

The cycles 13-15 artifacts are concentrated in these files:

| Area | Files |
|---|---|
| Security model | memory-centric-agentic/security_provenance_model.md, scripts/evaluate_security_provenance.py, scripts/plot_security_provenance.py, data/security_*.csv, data/security_*.png |
| Final synthesis | memory-centric-agentic/final_synthesis.md, scripts/synthesize_research_agenda.py, scripts/plot_synthesis.py, data/synthesis_*.csv, data/synthesis_*.png |
| Measurement plans | memory-centric-agentic/experiments/trajectory_reuse_measu<br>memory-centric-agentic/experiments/provenance_overhead_mea<br>memory-centric-agentic/experiments/cache_durable_risk_meas |
| Measurement harness | data/measurement_experiment_specs.csv, data/measurement_required_fields.csv, data/measurement_thresholds.csv, data/measurement_claim_update_matrix.csv, data/measurement_synthetic_probe_results.csv |
| Merge verification | memory-centric-agentic/experiments/dc005_merge_verificatio<br>tests/verify_dc005_merge_ready.py, data/dc005_merge_verification_results.csv |

Workspace totals after the manifest update are:

| Category | Count |
|---|---|
| Python scripts | 25 |
| Wolfram scripts | 4 |
| Total script lines | 7,376 |
| Markdown model/synthesis files | 16 |
| Experiment-plan and verification markdown files | 4 |
| CSV data/model files | 70 |
| Figures | 30 |

**Figure Inventory Used**

| Figure | Dimensions |
|---|---|
| data/security_risk_by_object.png | 1760 x 960 |
| data/security_mitigation_coverage.png | 1760 x 960 |
| data/security_architecture_tradeoff.png | 1920 x 960 |
| data/synthesis_architecture_matrix.png | 1920 x 960 |
| data/synthesis_agenda_priority.png | 1920 x 1120 |
| data/synthesis_claim_risk_map.png | 1600 x 960 |

**Validation Results**

The following checks were run during report preparation:

| Command | Result |
|---|---|
| python3 tests/verify_dc005_merge_ready.py | PASS, 10 checks. |
| python3 -m long_exposure.tools.org_check <workspace> | PASS. |

| Command | Result |
|---|---|
| `python3 -m long_exposure.tools.promise_check .` | FAIL due to ledger ordering: `M-EXP-1` has a validated fan-out clone event followed by in-progress parent integration without an intervening reopened event. |

The `promise_check` failure is reported as an audit-trail ordering issue. The DC-005 verifier and `org_check` passed, and the supplied audit report records the clone-0 and clone-2 branch deliverables as done. Clone 1 remains the explicit gap because it exited without a report.

**Source Reference Map**

| Report section | Source material |
|---|---|
| Abstract and introduction | Directive, restored cycles 10-12 context, `plan_of_record.md`, `final_synthesis.md` |
| Cycle 13 security results | Cycle 13 sessions, `security_provenance_model.md`, `evaluate_security_provenance.py`, `data/security_*.csv`, security figures |
| Cycle 14 synthesis results | Cycle 14 sessions, `final_synthesis.md`, `synthesize_research_agenda.py`, `data/synthesis_*.csv`, synthesis figures |
| Cycle 15 measurement harness | Cycle 15 researcher session, supplied fan-out audit report, experiment-plan markdown files, measurement CSVs, `dc005_merge_verification.md`, verifier output |
| References | `REFERENCES.md` |
| Implementation details | `MANIFEST.md`, workspace file inventory, CSV row counts, figure dimensions, validation commands |

**Cross-Cycle Dependency Map**

| Earlier artifact | Cycles 13-15 use |
|---|---|
| `data/runtime_ablation_results.csv` | Defines why hidden provenance/reuse fields collapse RAG and why hidden branch/verifier/durable fields collapse Option C. |
| `data/compression_object_queue_interactions.csv` | Prevents unsupported queue-help claims from being revived in the synthesis. |
| `data/calibration_deferred_constants.csv` | Names DC-003, DC-004, DC-005, and DC-006 as first-class measurement targets. |
| `memory-centric-agentic/trace_schema.md` | Supplies trace v2 fields and reveals production security-field gaps. |
| `memory-centric-agentic/runtime_prototype.md` | Provides the object-registry and replay-loop context for measurement plans. |
| `memory-centric-agentic/calibration_map.md` | Supplies sourced versus deferred calibration boundaries used by the synthesis. |